

Below are drop-in files/snippets to wire **Recommended Agents** and **Tag an Agent** into your Streamlit EVA Front. They assume the patched `base/s3_base.py` (with tags + precomputed neighbors support) and the offline jobs `reco_index.py` and `tags_engine.py` provided earlier.

Quick run order

1. Ensure `LOG_BUCKET` and `email` are set/available in the running session.
2. Run `python reco_index.py` once to create `agents/index/*` JSONs in S3.
3. (Optional) Run `python tags_engine.py` once for auto-tags.
4. Start EVA: `cd app && python -m streamlit run src/app.py`.

1) `src/ui_hooks.py` — UI helpers (Recommended + Tag editor)

```
# -*- coding: utf-8 -*-
from __future__ import annotations
import os
from typing import List, Dict
import streamlit as st

from base.s3_base import AgentManager

# ----- cached getters -----
@st.cache_data(ttl=300)
def _am(bucket: str, user_email: str) -> AgentManager:
    return AgentManager(bucket=bucket, user_email=user_email)

@st.cache_data(ttl=300)
def get_neighbors(bucket: str, user_email: str, agent_id: str, k: int = 8) -> List[str]:
    am = _am(bucket, user_email)
    return am.get_similar_agents(agent_id, k=k)

@st.cache_data(ttl=300)
def get_cluster_representatives(bucket: str, user_email: str, k: int = 12) -> List[str]:
    am = _am(bucket, user_email)
    return am.get_cluster_representatives(top=k)

@st.cache_data(ttl=300)
def get_tag_catalog(bucket: str, user_email: str) -> Dict[str, int]:
    am = _am(bucket, user_email)
    try:
        return am.list_tag_catalog()
    except Exception:
        return {}

# ----- renderers -----
def render_recommended_section(bucket: str, user_email: str,
```

```

current_agent_id: str | None = None):
    am = _am(bucket, user_email)
    st.subheader("Recommended Agents")

    # Recommendation mode selector
    rec_mode = st.selectbox(
        "Modo de recomendação",
        ["Similar a um agente", "Para você (diversificado por clusters)"]
    )

    if rec_mode == "Similar a um agente":
        all_ids = am.list_all_agents()
        if not all_ids:
            st.info("Nenhum agente encontrado.")
            return
        seed = st.selectbox("Escolha um agente de referência", all_ids)
        ids = am.get_similar_agents(seed, k=8)
    else:
        ids = am.get_cluster_representatives(top=12)

    if not ids:
        st.info("Sem recomendações ainda. Peça para executar o job 'reco_index.py'.")
        return

    # Optional tag filter
    cat = get_tag_catalog(bucket, user_email)
    tags = ["(todos)"] + sorted(cat)
    tag_sel = st.selectbox("Filtrar por tag", tags)

    for aid in ids:
        ag = am.get_agent(aid)
        if isinstance(ag, str):
            continue
        tg = am.get_tags(aid)
        if tag_sel != "(todos)" and tag_sel not in tg:
            continue
        with st.container(border=True):
            st.markdown(f"{ag.nome_agente}")
            if ag.desc:
                st.write(ag.desc[:220] + ("..." if len(ag.desc) > 220 else ""))
            if tg:
                st.caption("Tags: " + ", ".join(tg))
            st.button("Acessar", key=f"open-{aid}")

def render_tag_editor(bucket: str, user_email: str, agent_id: str | None = None):
    am = _am(bucket, user_email)
    st.subheader("Tag an Agent")
    ids = am.list_all_agents()

```

```

if not ids:
    st.info("Nenhum agente encontrado.")
    return
aid = st.selectbox("Selecione o agente", ids, index=0 if agent_id is None
else max(0, ids.index(agent_id)))
cur = am.get_tags(aid)
t = st.text_input("Tags (separe por vírgula)", value=", ".join(cur))
if st.button("Salvar tags"):
    tags = [x.strip() for x in t.split(",") if x.strip()]
    am.replace_tags(aid, tags)
    st.success("Tags atualizadas!")

```

2) Add the section to the EVA Store page

Open `src/paginas/_EVA_Store.py` (or the corresponding Store page) and insert near the top of the `main()` (or script) body:

```

import os
import streamlit as st
from ui_hooks import render_recommended_section, render_tag_editor

bucket = os.getenv("LOG_BUCKET")
email = st.session_state.get("email", "user@example.com")

st.markdown("## _EVA Store")

with st.expander("Recommended Agents", expanded=True):
    render_recommended_section(bucket, email)

with st.expander("Tag an Agent", expanded=False):
    render_tag_editor(bucket, email)

```

This creates two expandable blocks: one with recommendations, one with the tag editor. Both use the existing `AgentManager` bucket/email context exposed by `app.py`.

3) (Optional) Show recommendations on the Home page

In the Home/landing page script (e.g., `src/paginas/Menu_EVA.py` or the page that renders “Bem-vindo à EVA”), add:

```

import os
import streamlit as st
from ui_hooks import render_recommended_section

bucket = os.getenv("LOG_BUCKET")

```

```
email = st.session_state.get("email", "user@example.com")

st.markdown("### Recomendados para você")
render_recommended_section(bucket, email)
```

4) Tiny addition to `base/s3_base.py`

Ensure these two methods exist in `AgentManager` (they are present in the patched file I gave you; include if missing):

```
def list_tag_catalog(self) -> dict:
    key = f"{self.base_folder}tags/_catalog.json"
    try:
        obj = self.client.get_object(Bucket=self.bucket, Key=key)
        return json.loads(obj["Body"].read())
    except Exception:
        return {}

def get_cluster_representatives(self, top: int = 12) -> list[str]:
    key = f"{self.base_folder}index/cluster_representatives.json"
    try:
        obj = self.client.get_object(Bucket=self.bucket, Key=key)
        reps = json.loads(obj["Body"].read())
        return reps[:top]
    except Exception:
        return []
```

5) Offline jobs (reminder)

- Build recommendation index: `python reco_index.py`
- Generate initial tags (optional): `python tags_engine.py`

These write JSON artifacts into `agents/index/` and `agents/tags/` in S3 (same bucket pointed by `LOG_BUCKET`).

6) Dependencies

Add to `requirements.txt` if not present:

```
scikit-learn>=1.4
numpy>=1.24
```

(Only for the offline job; the UI uses the precomputed JSONs.)

That's it — after placing `ui_hooks.py` and updating the Store (and optionally Home) page, the EVA app will render a **Recommended Agents** block and a **Tag an Agent** editor, backed by your S3 YAML inventory.